



Esercitazione 5

Discrete Fourier Transform DTF (seconda parte)



Introduzione

Lo scopo dell'esercitazione è esplorare la rappresentazione in frequenza di segnali (DFT/IDFT), sperimentare la ricostruzione di segnali a partire da DFT **troncate** e osservare gli effetti pratici (smussamento, Gibbs, perdita di dettaglio) quando si rimuovono componenti spettrali. In breve: capire come manipolazioni nello spettro si riflettono nel dominio temporale.



Riassunto Esercitazione 5

In questa esercitazione si è analizzata la **Discrete Fourier Transform (DFT)** e la sua implementazione numerica tramite **Python (SciPy)**.

Gli step principali svolti sono stati:

- **DFT/IDFT**: sapere calcolare e interpretare la DFT (modulo & fase) e usare l'IDFT per ricostruire segnali.
- **Relazione tempo↔frequenza**: discontinuità nel tempo $\Rightarrow$ spettro a decadimento lento (molte componenti frequenziali); tagliare le alte frequenze produce smussamento e oscillazioni (Gibbs).
- **Troncamento dello spettro = filtraggio passa-basso grossolano**: porta perdita di dettaglio temporale e fenomeni di ringing attorno a discontinuità.
- **Leakage e risoluzione**: una sinusoide pura appare come picco stretto nello spettro; segmenti o transizioni brevi causano dispersione/spreading in frequenza.
- **Abilità pratica**: come normalizzare audio, salvare WAV, visualizzare primo n campioni e spettro normalizzato, confrontare spettro originale e troncato, e commentare gli effetti uditivi/visivi.

```
In [5]: import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
import scipy.fft as fft
from scipy.io import wavfile
import scipy
```



Esercizio 1

(a) Leggere il file `frequencyRepresentation.mat` usando:

```
x = np.squeeze(sio.loadmat('frequencyRepresentation.mat')['x'])
```

(b) Plot dei primi 500 campioni di `x`.

(c) Salvare `x` come audio WAV a 8000 Hz:

```
scipy.io.wavfile.write('sound1.wav', 8000, x)
```

(d) Calcolare la DFT di `x` e plot del suo modulo.

Domanda: In quale dei tre modi (time plot, ascolto, spettro) è più chiaro il numero di sinusoidi presenti?

```
In [3]: # Esercizio 1 - Analisi in frequenza di un segnale audio

print("Esercizio 1")

# (a) Lettura del segnale
x = np.squeeze(sio.loadmat('frequencyRepresentation.mat')['x'])
print(f"Raw: shape={x.shape}, dtype={x.dtype}")

# Pulizia / conversioni necessarie
# 1) Se è complesso -> prendo la parte reale
if np.iscomplexobj(x):
    print("Segnale complesso: prendo la parte reale.")
    x = np.real(x)

# 2) Se multi-canale -> converto a mono (media tra i canali)
if x.ndim > 1:
    # se è (N,1) o (1,N) squeeze in passato potrebbe aver già risolto,
    # qui gestiamo comunque casi con >1 canale
    if x.shape[0] == 1:
        x = x[0]
    elif x.shape[1] == 1:
        x = x[:, 0]
    else:
        print("Segnale multi-canale: converto a mono facendo la media dei canali.")
        x = x.mean(axis=1)

print(f"Dopo squeeze/mono/real: shape={x.shape}, dtype={x.dtype}")

# (b) Plot dei primi 500 campioni
plt.figure(figsize=(8,3))
plt.plot(x[:500])
plt.xlabel("Campioni")
plt.ylabel("Segnale")
plt.title("Primi 500 campioni del segnale (raw)")
plt.grid(True)
plt.show()
```

```
# Normalizzazione: portiamo in [-1, +1]
max_abs = np.max(np.abs(x))
print(f"Valore massimo assoluto (prima della normalizzazione): {max_abs:.6g}")

if max_abs == 0:
    # segnale nullo - niente da fare
    x_norm = x.astype(np.float32)
else:
    x_norm = (x / max_abs).astype(np.float32)

print(f"Valore massimo assoluto (dopo normalizzazione): {np.max(np.abs(x_norm)):.6g}")

# (c) Salvataggio .wav (int16 e float32)
Fs = 8000 # frequenza di campionamento richiesta
# 1) versione int16 (più compatibile con Lettori esterni)
x_int16 = (x_norm * 32767.0).astype(np.int16)
wavfile.write('sound1_int16.wav', Fs, x_int16)
print("Audio salvato come 'sound1_int16.wav' (int16, Fs=8000).")

# 2) versione float32 (qualità senza quantizzazione, alcuni player la supportano)
wavfile.write('sound1_float32.wav', Fs, x_norm)
print("Audio salvato come 'sound1_float32.wav' (float32, Fs=8000).")

# (d) Calcolo DFT e modulo (uso il segnale normalizzato float)
X = fft.fft(x_norm)
frequenze_normalizzate = np.arange(len(X)) / len(X)

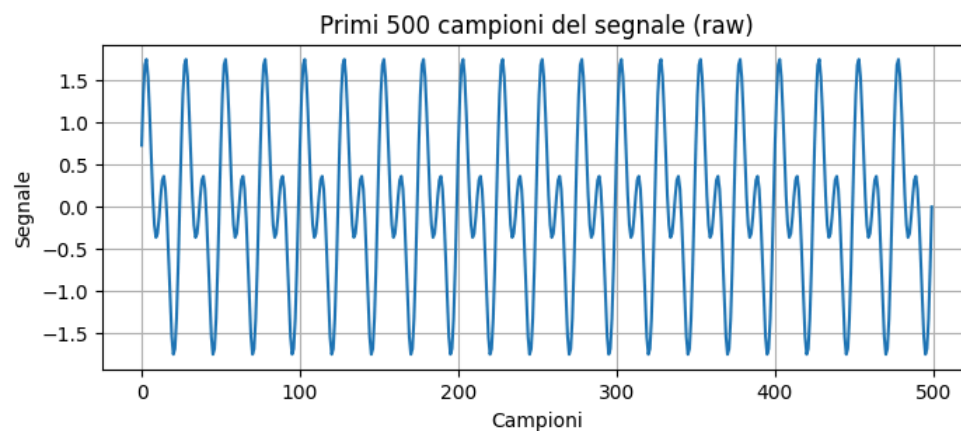
plt.figure(figsize=(8,3))
plt.plot(frequenze_normalizzate, np.abs(X))
plt.xlabel("Frequenze normalizzate")
plt.ylabel("Modulo della DFT")
plt.title("Spettro del segnale (DFT su segnale normalizzato)")
plt.grid(True)
plt.show()

# Domanda
print(" ? In quale dominio il numero di sinusoidi è più facilmente individuabile?")
print("- Time domain: difficile, le sinusoidi si sovrappongono")
print("- Audio: possibile intuire il numero di toni, ma non è chiaro")
print("- Spettro (DFT): ✅ più chiaro, ogni senoide appare come un picco netto")
```

Esercizio 1

Raw: shape=(4000,), dtype=float64

Dopo squeeze/mono/real: shape=(4000,), dtype=float64

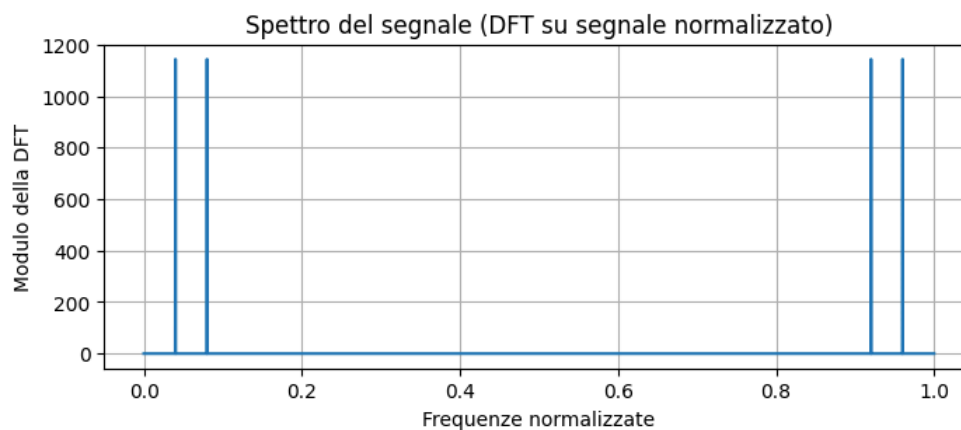


Valore massimo assoluto (prima della normalizzazione): 1.74915

Valore massimo assoluto (dopo normalizzazione): 1

Audio salvato come 'sound1_int16.wav' (int16, Fs=8000).

Audio salvato come 'sound1_float32.wav' (float32, Fs=8000).



? In quale dominio il numero di sinusoidi è più facilmente individuabile?

- Time domain: difficile, le sinusoidi si sovrappongono
- Audio: possibile intuire il numero di toni, ma non è chiaro
- Spettro (DFT): ✅ più chiaro, ogni senoide appare come un picco netto

COMMENTO.

Obiettivo: rappresentare un segnale in dominio tempo e frequenza per identificare periodicità e componenti principali.

Commento ai plot:

- Nel tempo si osservano periodicità o transitori che rivelano la natura del segnale.
- Lo spettro mostra picchi netti corrispondenti a sinusoidi dominanti; l'ampiezza dei picchi dà l'energia di ciascuna frequenza.
- Simmetria dello spettro dovuta al segnale reale.

Take-home message: un segnale complesso può essere scomposto in sinusoidi; i picchi dello spettro indicano le frequenze predominanti e i transitori generano larghezza e dispersione in frequenza.

✦ Esercizio 2

- (a) Considerare il segnale y di lunghezza 128 in cui la prima metà vale 1 e la seconda metà 0, e plot di y .
- (b) Calcolare la DFT di y .
- (c) Costruire i vettori $Y^{\wedge}(k)$ di lunghezza 128 con i primi $k+1$ coefficienti DFT non azzerati ($k=0,\dots,9$) e inversa. Plot di tutte le ricostruzioni in un'unica figura.

```
In [4]: # Esercizio 2 - Ricostruzione da DFT troncata

print("Esercizio 2")

### (a) Creazione del segnale y
# Segnale di lunghezza 128: primi 64 elementi a 1, resto 0
y = np.zeros(128)
y[:64] = 1

plt.figure()
plt.plot(np.arange(128), y)
plt.title("Esercizio 2(a) - Segnale a gradino")
plt.xlabel("Campioni")
plt.ylabel("Valore")
plt.grid(True)
plt.show()

### (b) Calcolo della DFT del segnale y
Y = fft.fft(y)

### (c) Ricostruzione da DFT troncata
# Preparazione per ricostruzione con coefficienti DFT parziali
num_el = 10 # Valori di k da 0 a 9
plt.figure(figsize=(10, 6))

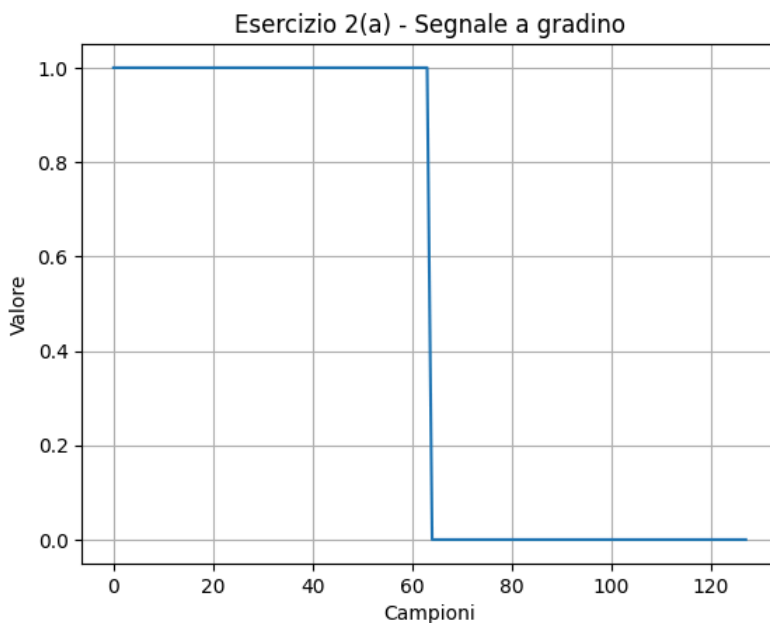
for k in range(num_el):
    # Vettore troncato: manteniamo solo i primi k+1 coefficienti
    Y_trunc = np.zeros_like(Y, dtype=complex)
    Y_trunc[:k+1] = Y[:k+1]

    # Ricostruzione tramite IDFT
    y_approx = fft.ifft(Y_trunc)
    y_approx_real = np.real(y_approx)

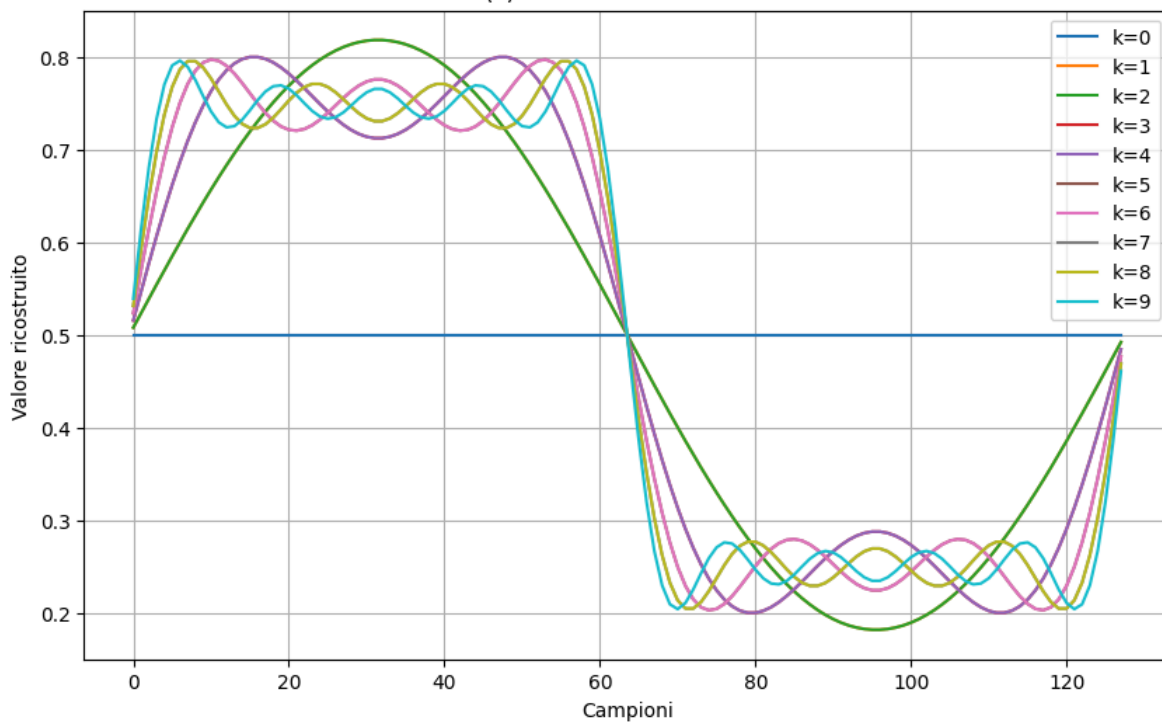
    # Plot della ricostruzione
    plt.plot(y_approx_real, label=f'k={k}')

plt.title("Esercizio 2(c) - Ricostruzioni con DFT troncata")
plt.xlabel("Campioni")
plt.ylabel("Valore ricostruito")
plt.legend()
plt.grid(True)
plt.show()
```

Esercizio 2



Esercizio 2(c) - Ricostruzioni con DFT troncata



COMMENTO:

Obiettivo: studiare l'effetto della troncatura dello spettro su un segnale a gradino.

Commento ai plot:

- Segnale originale: discontinuità netta.
- Con poche armoniche la ricostruzione diventa smussata e compaiono oscillazioni attorno alla discontinuità (Gibbs).
- Aumentando il numero di armoniche la ricostruzione migliora ma le oscillazioni non scompaiono del tutto.

Take-home message: un gradino richiede infinite armoniche; la troncatura comporta perdita di dettaglio e fenomeni di ringing.

✦ Esercizio 3

(a) Definire i segnali di lunghezza $N=1000$:

- $z1[n] = \sin(80\pi n / N)$
- $z2[n] = \sin(160\pi n / N)$
- $z3[n] = \sin(320\pi n / N)$

dove $n = 0, \dots, 999$.

(b) Concatenare in z i tre segnali con 1000 zeri di silenzio tra di essi (lunghezza totale 5000). Salvare come WAV ($fs=5000$) e ascoltare.

(c) Calcolare la DFT di z e plot del suo modulo.

(d) Definire Z_{approx} tagliando a zero tutti i coefficienti di Z tranne i primi 600 e gli ultimi 600, fare plot del modulo, inversa, salvare ($fs=5000$) e confrontare audio.

In [7]: # Esercizio 3 - Segmenti sinusoidali e filtraggio in frequenza

```
print("Esercizio 3")

### (a) Definizione dei segnali sinusoidali
N = 1000
t = np.arange(N)

# Tre sinusoidi a diverse frequenze
z1 = np.sin(80 * np.pi * t / N)
z2 = np.sin(160 * np.pi * t / N)
z3 = np.sin(320 * np.pi * t / N)

### (b) Concatenazione con silenzio
# Creiamo un segnale z di lunghezza 5000 con zeri tra i segmenti
z = np.zeros(5000)
z[0:1000] = z1
z[2000:3000] = z2
z[4000:5000] = z3

# Normalizzo e converto in int16 per WAV
def normalize_audio(x):
    x = x / np.max(np.abs(x)) # scala in [-1, 1]
    return (x * 32767).astype(np.int16)
```

```

z_int16 = normalize_audio(z)

# Salvataggio come file audio (frequenza campionamento 5000 Hz)
scipy.io.wavfile.write('sound2.wav', 5000, z_int16)
print("✅ Segnale originale salvato come 'sound2.wav'")

### (c) Calcolo della DFT e visualizzazione
Z = fft.fft(z)
frequenze_normalizzate = np.arange(len(Z)) / len(Z)

plt.figure(figsize=(10, 4))
plt.plot(frequenze_normalizzate, np.abs(Z))
plt.xlabel("Frequenze normalizzate")
plt.ylabel("|Z|")
plt.title("Esercizio 3(c) - Modulo della DFT di z")
plt.grid(True)
plt.show()

### (d) Approssimazione filtrando in frequenza
Z_approx = np.copy(Z)

# Azzeriamo tutti i coefficienti tranne primi 600 e ultimi 600
Z_approx[600:-600] = 0

# Visualizzazione spettro filtrato
plt.figure(figsize=(10, 4))
plt.plot(frequenze_normalizzate, np.abs(Z_approx))
plt.xlabel("Frequenze normalizzate")
plt.ylabel("|Z_approx|")
plt.title("Esercizio 3(d) - Modulo della DFT troncata (Zapprox)")
plt.grid(True)
plt.show()

# Ricostruzione tramite IDFT
z_approx = np.real(fft.ifft(Z_approx))

# Normalizzo e converto in int16 per WAV
z_approx_int16 = normalize_audio(z_approx)

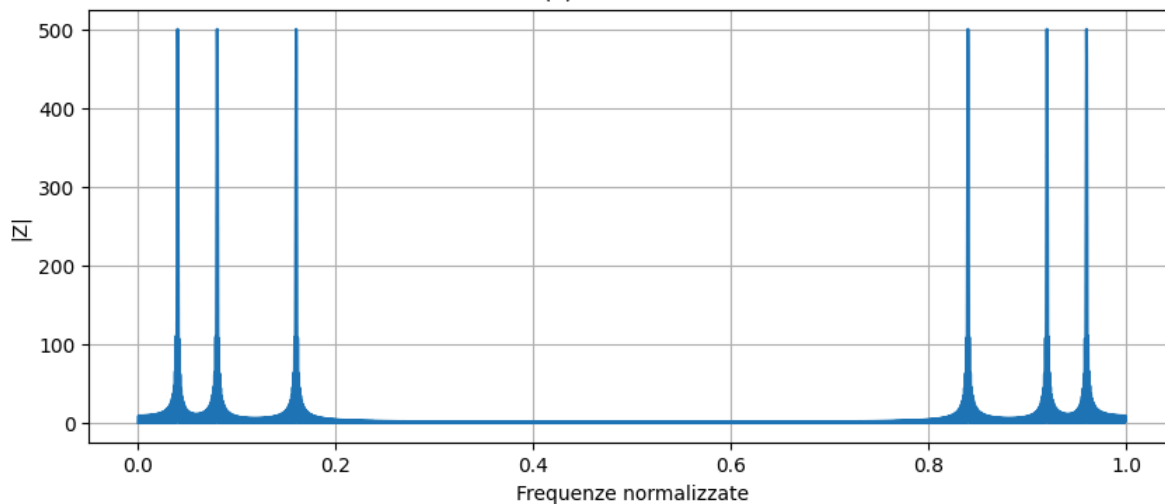
# Salvataggio del segnale ricostruito
scipy.io.wavfile.write('sound3.wav', 5000, z_approx_int16)
print("✅ Segnale ricostruito salvato come 'sound3.wav'")

```

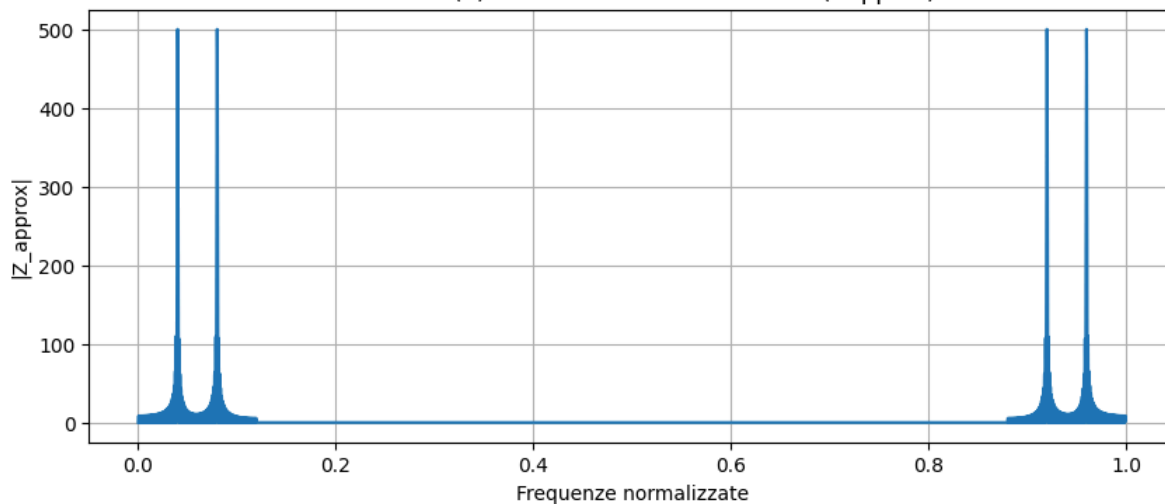
Esercizio 3

✅ Segnale originale salvato come 'sound2.wav'

Esercizio 3(c) - Modulo della DFT di z



Esercizio 3(d) - Modulo della DFT troncata (Zapprox)



✔ Segnale ricostruito salvato come 'sound3.wav'

COMMENTO.

Obiettivo: analizzare lo spettro di sinusoidi concatenate e l'effetto del filtraggio in frequenza.

Commento ai plot:

- Lo spettro mostra picchi corrispondenti alle frequenze delle sinusoidi.
- La finestratura temporale dei segmenti produce allargamento dei picchi (leakage).
- Con spettro troncato alcune componenti scompaiono, la ricostruzione perde toni e i transitori risultano smussati.

Take-home message: i picchi nello spettro rappresentano le sinusoidi originali; il leakage riflette la limitazione temporale; eliminare componenti spettrali altera contenuto tonale e dettagli del segnale.